Introduction aux Bases de Données Relationnelles et Requêtes SQL (Norme ISO/ANSI)

Thème principal : Bases de Données Relationnelles, Modèle Relationnel, Clés et Requêtes SQL

Synthèse exécutive : Ce cours pose les fondations indispensables de la ressource R1.05 dispensée en BUT 1 Informatique à l'IUT de Bayonne et du Pays Basque. L'objectif pédagogique est de comprendre la transition critique depuis les fichiers plats et tableurs (Excel) vers les Systèmes de Gestion de Bases de Données Relationnelles (SGBD). Le cours démontre comment la décomposition en tables reliées par des clés primaires et étrangères élimine la redondance de données et les anomalies de mise à jour tout en garantissant l'intégrité référentielle. Il enseigne également la rédaction de requêtes d'extraction SQL normalisées (norme ISO/ANSI) en utilisant la syntaxe explicite INNER JOIN ... ON au détriment du produit cartésien obsolète.

Du Tableur Flat File au Modèle Relationnel

Dans les applications informatiques réelles, l'utilisation d'une feuille de calcul plate (type Excel) montre immédiatement ses limites dès que le volume de données grandit ou que des entités interagissent. Si l'adresse d'un client est répétée sur 50 lignes de commandes, un simple déménagement exige de modifier 50 lignes distinctes. Si une seule ligne est manquée, la base d'informations devient contradictoire. Le modèle relationnel résout ce problème à la racine en séparant les entités dans des tables distinctes ('Client' et 'Commande') et en établissant des liens logiques entre elles sans duplication de l'information principale.

- Redondance de données : Duplication inutile d'une même information à plusieurs endroits d'un fichier plat ou d'une base de données.
- Anomalie de mise à jour : Incohérence grave survenant lorsqu'une information répétée est modifiée sur certaines lignes mais oubliée sur d'autres.
- SGBD : Logiciel qui administre l'accès aux données tout en garantissant la cohérence, la sécurité, l'accès concurrentiel et la persistance.

Cas concret enseignant : *L'enseignant a illustré cette problématique avec l'analogie du système d'information bancaire : si une banque enregistrait le nom, le prénom et l'adresse du titulaire sur chaque ligne de virement ou de transaction bancaire, un simple déménagement exigerait de repasser sur des milliers d'opérations passées, créant un risque critique d'erreur de saisie et saturant inutilement les serveurs de stockage.*

Clés Primaires, Clés Étrangères et Intégrité Référentielle

Afin qu'un SGBD puisse identifier chaque enregistrement sans aucune ambiguïté, toute table relationnelle doit définir au moins une Clé Primaire. Cette clé sert d'ancre. Pour créer une association entre deux tables (par exemple lier un Étudiant à son Groupe de TD), on n'insère pas la description du groupe dans l'étudiant : on se contente d'y stocker la clé primaire du Groupe en tant que Clé Étrangère. Le moteur de base de données surveille ensuite cette liaison : toute tentative d'affecter une valeur de clé étrangère qui n'existe pas dans la table d'origine est rejetée.

- Clé Primaire (Primary Key - PK) : Attribut ou ensemble d'attributs identifiant chaque ligne (n-uplet) de façon unique et obligatoirement non nulle (NOT NULL).
- Clé Étrangère (Foreign Key - FK) : Attribut présent dans une table qui fait référence directe à la clé primaire d'une autre table pour matérialiser un lien logique.
- Intégrité Référentielle : Règle métier automatisée par le SGBD qui interdit à une clé étrangère de pointer vers un identifiant inexistant dans la table mère.

Cas concret enseignant : *Exemple du Numéro de Sécurité Sociale (NIR) et du numéro d'étudiant à l'IUT de Bayonne : Même si deux étudiants portent exactement le même nom et le même prénom (ex: deux Jean Dupont au Pays Basque), leur identifiant unique empêche toute confusion ou doublon au sein du système d'information.*

Extraction de Données en SQL ISO/ANSI : Projection, Filtrage et Jointures Explicites

Pour croiser des informations réparties sur plusieurs tables, la norme ISO/ANSI impose une syntaxe explicite utilisant la clause INNER JOIN table2 ON table1.fk = table2.pk. Cette forme isole la définition de la jointure (dans la clause ON) des critères de filtrage des données (dans la clause WHERE). La méthode ancienne consistant à lister les tables séparées par des virgules dans le FROM et à placer la condition de jointure dans le WHERE est un produit cartésien implicite déconseillé, moins optimisé et propice aux erreurs de logique.

- Langage Déclaratif : Le SQL exprime CE QUE l'on souhaite obtenir et non l'enchaînement algorithmique d'instructions (COMMENT).
- Projection (SELECT) : Choix explicite des colonnes à extraire et à afficher dans le résultat.
- Filtrage (WHERE) : Application de critères logiques pour cibler uniquement les n-uplets d'intérêt.
- Jointure Explicite (INNER JOIN ... ON) : Fusion normalisée ISO/ANSI de deux tables basée sur la correspondance stricte entre Clé Étrangère et Clé Primaire.

Cas concret enseignant : *Mise en garde stricte du professeur pour les partiels et examens : 'N'utilisez jamais la syntaxe obsolète du produit cartésien avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite INNER JOIN ... ON. Le produit cartésien implicite fait perdre des points en partiel et est formellement interdit en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on pas utiliser SELECT * dans un code applicatif ou lors d'un examen ?
Réponse du professeur : Parce que SELECT * récupère l'intégralité des colonnes sans distinction, ce qui alourdit les transferts réseau, rompt la compatibilité si la structure de la table évolue, et montre un manque de rigueur dans la projection des données demandées.

Question étudiant : Que se passe-t-il si j'essaie de supprimer un groupe qui contient encore des étudiants rattachés ?
Réponse du professeur : Le SGBD bloque la suppression en raison de la contrainte d'intégrité référentielle. Il protège ainsi la base contre les orphelins de données (des étudiants pointant vers un groupe inexistant).

■ Exercice & Démonstration d'Application

Énoncé : On considère le schéma relationnel suivant : - Etudiant (id_etudiant, nom, prenom, #id_groupe) - Groupe (id_groupe, nom_groupe, promo) Écrire la requête SQL conforme à la norme ISO/ANSI permettant d'extraire le nom et le prénom des étudiants appartenant à la promotion 'BUT1-INFO' et inscrits dans le groupe nommé 'A1'.

Démarche de résolution :
Étape 1 : Identifier les tables nécessaires (Etudiant et Groupe) et définir des alias clairs (e pour Etudiant, g pour Groupe). Étape 2 : Établir la jointure explicite normalisée reliant la clé étrangère e.id_groupe à la clé primaire g.id_groupe via INNER JOIN Groupe g ON e.id_groupe = g.id_groupe. Étape 3 : Poser les conditions logiques de filtrage dans la clause WHERE (g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'). Étape 4 : Définir la projection explicite des colonnes demandées dans la clause SELECT (e.nom, e.prenom).

```
SELECT e.nom, e.prenom
FROM Etudiant e
INNER JOIN Groupe g ON e.id_groupe = g.id_groupe
WHERE g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1';
```

Résultat : La requête retourne la liste exacte des noms et prénoms des étudiants inscrits dans le groupe A1 de la promo BUT1-INFO. La requête respecte la norme ANSI, évite le produit cartésien implicite et présente des performances d'exécution optimales.

■ Auto-Évaluation Flash d'Examen

Q : Quelles sont les deux contraintes fondamentales et obligatoires d'une Clé Primaire (Primary Key) ?

R : L'unicité de chaque valeur et l'interdiction de la valeur nulle (NOT NULL).

Q : Pourquoi la syntaxe 'FROM TableA, TableB WHERE TableA.id_b = TableB.id' fait-elle perdre des points en partiel de R1.05 ?

R : Parce qu'il s'agit d'un produit cartésien implicite obsolète qui mélange la logique de structure et la logique de filtrage, au mépris de la norme ISO/ANSI moderne.

Q : Qu'est-ce que la contrainte d'intégrité référentielle dans un SGBD relationnel ?

R : C'est une règle automatique garantissant qu'une clé étrangère pointe toujours vers une clé primaire existante, empêchant ainsi les données orphelines.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765